Studio Nx

JS 113 — 루프

2026-10-14

JS 113 — 루프 (`Transport.setLoop()` · `planSchedule` 루프)

항목	내용
파일	`studionx/frontend/static/js/audio/scheduler.js` (`planSchedule` 인자 추가 + `Transport.setLoop`)
작업	S1-05 오디오 엔진 기초: 열 번째 함수 (FR-204 루프)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-05-audio-engine`
다음	S1-05 마무리(전체 회귀·커밋 결정). 이후 S1-06

1. 하는 일

재생 위치가 루프 끝(tick)에 닿으면 시작(tick)으로 되돌아갑니다. 시간(초)은 계속 흐르고 tick만 되돌아갑니다.

대상	하는 일
`Transport.setLoop(start, end, enabled)`	루프 구간·on/off 설정. 재생 중이면 지금 위치로 다시 예약
`planSchedule(..., loop)`	`{start, end}`를 받으면 끝을 지나는 단계의 tick을 시작으로 되돌림
`Transport.positionTick()`	루프가 켜져 있고 끝을 지나면 시작 기준으로 되돌린 tick을 돌려줌

```
transport.setLoop(0, 3840, true);   // 0~3840 tick(1마디) 구간 반복
transport.start(0);
```

2. 코드

`planSchedule`에 `loop` 인자를 더했습니다(없으면 지금까지와 같음).

```
  if (loop != null && (!(loop.end > loop.start))) throw new RangeError('loop 은 start < end 여야 합니다:
' + loop.start + ' ~ ' + loop.end);

  const loopLen = loop != null ? loop.end - loop.start : 0;
  function wrap(t) {
    if (loop == null) return t;
    return t >= loop.end ? loop.start + ((t - loop.start) % loopLen) : t;
  }
  // 반복 안:
  //   const rawTick = tick + stepTicks;
  //   const delta = f(rawTick) - f(tick);   // f = ticksToSecondsWithChanges
  //   tick = wrap(rawTick);
  //   time += delta;
```

`Transport`에 구간 필드와 `setLoop`을 더했습니다.

```
  setLoop(startTick, endTick, enabled) {
    if (startTick != null) this.loopStart = startTick;
    if (endTick != null) this.loopEnd = endTick;
    if (enabled != null) this.loopEnabled = !!enabled;
    if (this.loopEnabled && !(this.loopEnd > this.loopStart)) throw new RangeError('loopEnd > loopStart
```

```
여야 합니다: ' + this.loopStart + ' ~ ' + this.loopEnd);
    if (this.playing) {
      const from = this._alignUp(this.positionTick());
      this._clearTimer();
      this._anchor(from, this.clock.now() + this.startDelay);
      this.pump();
      this._scheduleTimer();
    }
    return this;
  }
  _loopRegion() {
    if (this.loopEnabled && this.loopEnd > this.loopStart) return { start: this.loopStart, end: this.loo
pEnd };
    return null;
  }
  // positionTick: 구한 tick 이 끝을 지나면 시작 기준으로 되돌림
  //   const loop = this._loopRegion();
  //   if (loop !== null && tick >= loop.end) return loop.start + ((tick - loop.start) % (loop.end - loo
p.start));
```

최신 문법으로 쓰면 (공부용)

```
const wrap = t => (loop == null || t < loop.end) ? t : loop.start + ((t - loop.start) % (loop.end - loo
p.start));
setLoop(startTick, endTick, enabled) { /* … */ return this; }
```

최신 문법	실제 코드	뜻
기본 매개변수 · 화살표 함수	`function wrap(t) {…}`	작은 도우미 함수를 안에서 정의합니다
`??=`	`if (enabled != null) this.loopEnabled = !!enabled;`	값이 있을 때만 바꿉니다(전통 문법)
`String.prototype`·템플릿	`'…' + a + ' ~ ' + b`	글자와 값을 잇습니다

3. 왜 이렇게

선택	이유
되돌림을 tick 공간의 나머지로	소리 시간(초)은 계속 커지고 tick만 접힙니다. 오디오 시각은 한 방향으로 흘러야 하기 때문입니다
끝을 지나는 단계의 걸린 시간은 "끝 직전" 기준	`delta = f(tick+step) - f(tick)` 로 계산해, 되돌아간 뒤에도 그 단계가 자연스럽게 이어집니다. 그래서 루프를 한 바퀴 도는 시간이 곡 템포와 맞습니다
`planSchedule` 이 계산을 담당	루프는 예약 목록을 만드는 일이라 순수 함수에 넣어야 Node로 시험할 수 있습니다
`Transport` 는 구간만 넘김	`_loopRegion()` 이 "켜져 있고 길이가 있으면" 구간을 돌려줘, 꺼졌거나 잘못된 값이면 `null` 이 되어 지금까지와 같게 돕니다
재생 중 `setLoop` 은 다시 예약	커서가 옛 구간 기준으로 앞서 있을 수 있어, 지금 위치로 커서를 되돌려 새 구간을 반영합니다 (`setTempo` 와 같은 방식, 최대 한 단계 앞으로 살짝 뛺)
멈춰 있을 때는 예약 안 함	위치만 바꾸고, 다음 `start`·`resume` 이 새 구간을 씁니다
아직 잡힌 창은 옛 구간대로	재생 중 설정하면 최대 lookahead(0.1초) 안의 이미 예약된 소리는 그대로 울립니다(문서에 남김)

선택	이유
시작이 구간보다 뒤여도 동작	`wrap` 은 `t >= loop.end` 일 때만 접으므로, 재생 위치가 구간에 들어와 끝을 지나면 반복이 시작됩니다. 템포 변경점이 구간에 걸쳐 있으면 오차가 날 수 있음(프로토타입 한계)

4. 시험 (Node에서 직접 실행, 27개 통과 · 가짜 시계·가짜 타이머)

확인	결과
`planSchedule` 루프: 16단계(2초)마다 tick 0으로 되돌아감, 시간 간격은 0.125초로 연속	✓
구간 안쪽 시작(`loop {960,3840}`)에서 시작해도 960으로 되돌아감, 모든 tick이 구간 안	✓
`nextTick`·`nextTime` 커서가 루프를 반영해 이어짐	✓
루프 없으면 지금까지와 같음(선형)	✓
루프 값 오류(start=end, start>end) 시 `RangeError`	✓
재생 중 tick 720 다음이 0으로 되돌아감, 시간은 계속 증가, 위치가 구간 안	✓
루프를 끄면 선형으로 이어짐	✓
멈춰 있을 때 `setLoop` 은 예약 안 함, 필드만 설정	✓
재생 중 루프 켜면 이후 예약이 구간을 따름, 위치가 구간 안	✓
`setLoop` self 반환, 생성 시 잘못된 루프는 `RangeError`	✓
회귀: 107(26)·108(23)·109(21)·110(18) 모두 통과	✓